However, as your spec suite grows and starts taking longer to complete, it can be nice to see failure details as soon as they occur. That way, you can start investigating a failure while the rest of the suite continues to run.

We've built a custom formatter for you that makes this change to RSpec's output. In this section, you're going to install the formatter and configure RSpec to use it. Afterward, we'll dig into how the formatter works.

Setting Up the Formatter

The new formatter is called rspec-print_failures_eagerly. We've made it available as a gem, but you're going to clone its source onto your machine instead of using gem install. This way, you'll be able to use it for all projects on your machine, rather than just the ones that mention it in their Gemfile.

First, clone the formatter's source code into your home directory:

```
$ git clone https://github.com/rspec-3-book/rspec-print_failures_eagerly.git
```

Now, to load the new library for every RSpec project, put the following content into a file called .rspec in your home directory:

09-configuring-rspec/02/configuring_rspec/.rspec

```
-I<%= ENV['HOME'] %>/rspec-print_failures_eagerly/lib
--require 'rspec/print_failures_eagerly'
```

Since you won't be relying on Bundler or RubyGems to manage Ruby's $LOAD_PATH, you'll have to do so here. The first line, starting with -I, makes the formatter's code available to RSpec. The second line actually loads the library.

RSpec supports the ERB (embedded Ruby) template syntax in this file, and we're using it to read the value of the HOME environment variable.[3]

With the setup done, you can run rspec and see the formatter in action. Here's what the output looks like on a suite with two passing examples and two failures:

```
$ rspec a_spec.rb
F

  1) A group with a failure has an example that fails
     Failure/Error: expect(1).to eq 2

       expected: 2
            got: 1

       (compared using ==)
     # ./a_spec.rb:3:in `block (2 levels) in <top (required)>'
```

3. https://codingbee.net/tutorials/ruby/ruby-the-erb-templating-system